

LIFESPHERE

Health · Safety · Automation · Care

An AI-Powered Autonomous Caregiving Ecosystem

Complete Technical Research and Build Document

Project Title: LifeSphere: AI-Powered Autonomous Caregiving
Ecosystem

Team: Team Fälschen

Lead Researcher: Faiyaz Bin Iqbal

“Forged. Wired. Perfected”

LIFESPHERE Team Fälschen

Team Fälschen

Faiyaz Bin Iqbal

Roll: 1219 | ID: 250485 | Class: VII | Section: L

Institution: Birshreshtha Munshi Abdur Rouf Public College

As the sole engineer and researcher behind the LifeSphere project, Faiyaz Bin Iqbal independently executed its entire technical scope, from conceptual research to full-stack deployment. He conducted all core research and designed every subsystem architecture, including documenting biosensor integration methodologies and independently sourcing and curating all biomedical datasets. Driving the project's artificial intelligence, he developed the trimodal LLM pipeline, built and trained machine learning models, and prepared the specific data pipelines for Vital32's LSTM-based predictive health models. His software engineering contributions encompass architecting the complete infrastructure, developing autonomous agent systems, engineering the embedded backend pipeline to manage cross-subsystem data transfer, building the Firebase real-time communication layer, and designing and launching the documentation web platform. Rounding out the project's physical and visual capabilities, he programmed full computer vision systems, designed all PCB layouts, engineered the complete robotics kinematics and manipulation systems, and rigorously tested and validated the autonomous medication dispensing and dosage mechanisms.

Contents

Contents	3
Abstract	4
1 Introduction and Motivation	5
2 System Architecture Overview	6
2.1 Ecosystems Overview & Functionality	6
2.2 How the Unified Ecosystem Helps Humanity	7
2.3 Distributed Intelligence Model	8
2.4 Shared Biological State Object	8
2.5 Inter-Subsystem Communication Protocol	8
2.6 Multi-Timescale Inference Architecture	8
2.7 Fault Tolerance and Graceful Degradation	9
2.8 Security and Data Integrity	9
3 Vital32: Complete Build Guide	10
3.1 Complete Component List	10
3.2 Wiring and Pin Configuration	10
3.3 Firmware Architecture	11
3.4 Fall Detection Algorithm	11
3.5 HRV Calculation	12
4 HollowRover: Complete Build Guide	13
4.1 Complete Component List	13
4.2 ESP32 Peripheral Controller Wiring	14
4.3 ROS2 Package Structure	14
4.4 SLAM Configuration	15
4.5 UWB Localization	15
4.6 Drivetrain: Rocker Suspension	15
4.7 HollowRover Robotic Arm	16
5 SphereAI: Implementation Guide	17
5.1 Software Dependencies	17
5.2 TriAgent Fusion Logic	17
6 EntryGuard and HollowCore: Implementation Guide	18
6.1 EntryGuard Hardware per Node	18
6.2 HollowCore Wiring	18
6.3 HollowCore Financial Anomaly Detection & Biometric Security	18
7 Firebase Backend Architecture	20
7.1 Firebase Project Setup	20
7.2 ESP32 Firebase Write (Vital32/EntryGuard)	20
7.3 Python Firebase Read (SphereAI/HollowRover)	20

8	AI Model Pipeline: Training and Deployment	21
8.1	Isolation Forest Anomaly Detector	21
8.2	Pre-Seizure LSTM	21
8.3	SphereAI Emotional State Classifier	22
8.4	Model Performance Summary	22
9	Mathematical Foundations	23
9.1	Biometric Signal Processing	23
9.2	Isolation Forest Anomaly Score	23
9.3	Medication Adherence Entropy	23
9.4	UWB Trilateration	23
10	Prescription OCR and Automated Medication Reminders	24
10.1	System Overview	24
10.2	OCR Pipeline and Data Extraction	24
10.3	Reminder Schedule Integration	25
10.4	Firebase Medication Schedule Schema	25
11	Absence Mode and Autonomous Household Management	26
11.1	Absence Detection	26
12	Remote Guardian and Digital Safety Integration	27
12.1	Remote User Configuration	27
12.2	AI-Powered Content Safety	27
13	Full Bill of Materials	28
13.1	Vital32	28
13.2	HollowRover	29
13.3	SphereAI	29
13.4	EntryGuard (×2 Nodes)	30
13.5	HollowCore	30
13.6	System Total	30
14	Conclusion	31
	References	32

Abstract

LifeSphere is a fully integrated, AI-powered autonomous caregiving ecosystem comprising five interconnected subsystems: Vital32, HollowRover, SphereAI, EntryGuard, and HollowCore. Collectively, they function as a single synchronized intelligence capable of monitoring, predicting, and responding to the user across four simultaneous dimensions: medical, physical, emotional, and environmental, without requiring continuous human supervision or explicit user input.

Vital32 is the primary biosensing layer, implemented on ESP32 microcontroller architecture with a 16-sensor array acquiring ECG waveforms, infrared skin temperature, accelerometer and gyroscope data for fall and tremor detection, bioelectrical impedance for tissue hydration quantification, galvanic skin response for sympathetic nervous system activity monitoring, and near-infrared tissue perfusion measurements for wound healing assessment. An onboard AI pipeline combining Isolation Forest anomaly detection and LSTM sequential modeling enables immune activation detection 24 to 48 hours prior to symptom onset, pre-seizure probability estimation 30 minutes before onset, and continuous biological age estimation through multi-sensor data fusion. All biometric data is transmitted in real time to Firebase Realtime Database, serving as the shared state input for all downstream subsystems.

HollowRover is the autonomous physical response unit operating on ROS2 with SLAM-based floor mapping and UWB precision positioning accurate to within 10 centimeters. Sensor fusion across TOF LiDAR, IMU, ultrasonic arrays, and thermal camera enables dynamic obstacle avoidance under variable conditions. A dedicated NPU on the D-Robotics RDK X5 compute board allows AI inference and SLAM navigation to run on separate silicon, eliminating CPU contention. HollowRover executes biomarker-triggered action protocols autonomously including medication dispensing, fall response dispatch, pre-seizure cushioning deployment, and cortisol-spike deescalation without manual command input. A 6-DOF robotic arm provides caregiving assistance (retrieving dropped objects, handing items to the user) and serves as a passive physical barrier during security response events.

SphereAI implements a trimodal emotional state classification architecture fusing NLP sentiment analysis, vocal prosody extraction, and live Vital32 biomarker feeds via a dynamic weighted meta-classifier. This enables early cognitive decline detection and longitudinal emotional state modeling. An Islamic integration framework provides salah reminders, stress-triggered adhkar, and a Ramadan mode restructuring medication and hydration protocols around fasting intervals.

EntryGuard implements perimeter security through PIR, ultrasonic, fire, and gas sensors at all entry points with face recognition cross-referenced against registered profiles. HollowCore maps physiological and emotional state variables directly to lighting, thermal, appliance, and digital safety outputs via Wi-Fi and MQTT protocol. LifeSphere does not require the user to interact with technology. It requires technology to respond to the user's biology.

1 Introduction and Motivation

Bangladesh is home to over 15 million elderly citizens, the majority of whom remain at home while family members work full-time. Every day, thousands forget critical medication, sustain falls with no one nearby to respond, and experience silent physical and emotional deterioration that goes undetected until it becomes life-threatening. Conditions detectable weeks in advance remain untreated due to limited healthcare accessibility, inadequate emergency response infrastructure, and the fundamental inability of current smart devices to act on the data they collect.

Problem Statement & Hypothesis

An elderly person living alone has no system that simultaneously monitors their health, responds to emergencies physically, provides emotional support, and protects their home, all autonomously, without requiring any action from the user.

Hypothesis: A fully integrated, bioresponsive caregiving ecosystem provides superior caregiving outcomes compared to any combination of existing single-function devices.

LifeSphere unifies health monitoring, autonomous robotics, emotional AI, home security, and environmental automation into a single organism sharing one biological state backbone. This document provides complete specifications sufficient to replicate the entire system from scratch, including circuit diagrams, firmware architecture, ROS2 package structure, AI model training procedures, and Firebase schema definitions.

Impact on Humanity

LifeSphere addresses the global crisis of elderly isolation and inadequate care. By autonomously bridging the gap between biological signals and physical intervention, it preserves human dignity, prevents preventable fatalities, and ensures that the most vulnerable members of society are never truly alone.

Deployment Scope

HollowRover is designed for single-floor deployment, consistent with industry-standard mobile caregiving and service robots (e.g., Amazon Astro, hospital delivery robots). Stair-climbing requires legged locomotion architectures that fall outside the cost and complexity scope of this build. All navigation, SLAM mapping, and physical response protocols are engineered for flat-floor indoor environments with standard doorway thresholds and uneven surfaces handled via passive rocker suspension.

2 System Architecture Overview

LifeSphere operates as a single distributed intelligence system across five hardware nodes sharing one unified data backbone. The architecture follows a publish-subscribe model with Firebase Realtime Database as the central biological state synchronization layer. Vital32 continuously writes physiological data to the shared state object. All remaining subsystems subscribe to relevant state fields and execute responses within a target latency of 150 milliseconds.

2.1 Ecosystems Overview & Functionality

Vital32: The Sensing Layer

How it Works: Vital32 continuously monitors the user’s physiological state using a 16-sensor array. It processes raw ADC readings, applies signal processing like HRV calculation, and feeds the data into multi-timescale AI models.

How it is Used: Worn by the user, it operates invisibly, requiring no interaction. It acts as the biological heartbeat of the entire ecosystem.

Humanity Impact: By shifting healthcare from reactive to proactive, Vital32 detects anomalies and predicts medical emergencies (like seizures or immune activation) hours or days before they become critical, saving lives through early intervention.

Subsystem	Primary Function	Core Hardware	AI Stack
Vital32	Continuous biometric monitoring and home automation trigger	ESP32, MAX30102, MPU6050, MLX90614, BME688, BIA circuit	LSTM, Isolation Forest, biological age model
HollowRover	Physical response, medicine delivery, security patrol, passive barrier deterrence	D-Robotics RDK X5 (4GB), STL-19P TOF LiDAR, DW1000 UWB, MLX90640 thermal, MG996R arm	SLAM, Nav2, biomarker-triggered protocol engine, DeepFace
SphereAI	Trimodal emotional intelligence and cognitive monitoring	Microphone, speaker, Whisper STT, OLED via rover	NLP + voice prosody + biomarker fusion classifier
EntryGuard	24/7 intrusion, fire, and gas detection at all entry points	PIR, MQ-2 gas, KY-026 fire, ESP32 nodes	DeepFace recognition, threat classification
HollowCore	Biology-responsive home and financial automation	Wi-Fi relay modules, MQTT broker, ESP32	Biomarker-to-environment mapping, anomaly detection

HollowRover: The Physical Response Unit

How it Works: Operating on ROS2 with SLAM and UWB positioning, HollowRover maps the home and navigates autonomously. It subscribes to Firebase and executes physical protocols when specific biomarker thresholds are crossed. A 6-DOF robotic arm provides caregiving assistance and passive security barrier functions.

How it is Used: It roams the home dispatching medication, responding to falls, retrieving dropped objects, and patrolling. It bridges the gap between digital alerts and physical action.

Humanity Impact: Ensures that no one remains on the floor after a fall and that critical medications are never missed. It provides a physical presence that purely digital systems cannot offer.

SphereAI: The Emotional Intelligence

How it Works: SphereAI fuses NLP sentiment analysis, vocal prosody, and live biomarkers using a dynamic meta-classifier to assess emotional and cognitive states. It listens and processes, adapting its weights based on prior emotional context.

How it is Used: It engages in natural conversation, provides verbal companionship, and triggers cultural/spiritual reminders like Salah or Adhkar based on emotional need.

Humanity Impact: Fights the silent epidemic of elderly loneliness and cognitive decline. It provides empathetic interaction and early detection of neurological deterioration.

EntryGuard: Perimeter Security

How it Works: Deploys multi-sensor nodes (PIR, ultrasonic, fire, gas) at all entry points. Uses facial recognition to distinguish between registered occupants and intruders, publishing alerts to Firebase.

How it is Used: Operates continuously as an invisible security layer. It arms automatically and alerts only when genuine threats are detected.

Humanity Impact: Provides peace of mind and physical safety for vulnerable individuals living alone, protecting against both environmental hazards and intrusions.

HollowCore: Biology-Responsive Automation

How it Works: Maps the user's physiological and emotional state directly to the home environment via Wi-Fi relays and MQTT. If the user is stressed, it dims lights; if cold, it adjusts the AC. It also monitors financial transactions for anomalies.

How it is Used: Operates entirely in the background. The home becomes a living organism that adapts to the user's biology without switches or commands.

Humanity Impact: Creates a therapeutic environment that reduces stress and enhances comfort, making the home an active participant in caregiving rather than a passive container.

2.2 How the Unified Ecosystem Helps Humanity

While individual subsystems target specific needs, the unified LifeSphere ecosystem creates a paradigm shift in caregiving. Traditional smart devices operate in silos—a smart-watch tracks heart rate, a smart speaker sets reminders, a security camera records video.

LifeSphere unifies these into a single biological state backbone. When Vital32 detects a cortisol spike, SphereAI adjusts its tone, HollowCore alters the lighting, and HollowRover prepares a stress-relief protocol. This multi-dimensional, synchronized response ensures that the user is cared for holistically—medically, physically, emotionally, and environmentally. By removing the need for the user to interact with technology, LifeSphere serves those who need care the most but are least capable of managing complex devices.

2.3 Distributed Intelligence Model

Each subsystem maintains local processing capability while simultaneously contributing to and consuming from the shared Firebase state object. This architecture ensures no single point of failure exists for user monitoring. If HollowRover is charging, Vital32 still monitors. If one ESP32 node fails, remaining nodes continue operating independently.

2.4 Shared Biological State Object

The core architectural primitive of LifeSphere is the biological state object: a continuously updated JSON document on Firebase containing the user’s complete physiological, emotional, and environmental profile.

Listing 1: Firebase JSON Schema

```
{
  "lifesphere": {
    "vital32": {
      "heart_rate": 72, "spo2": 98.2, "skin_temp": 36.4,
      "hrv_rmssd": 42.1, "hrv_lf_hf": 1.8, "gsr": 0.34,
      "bio_age": 68.4, "anomaly_score": 0.12,
      "seizure_prob": 0.03, "cortisol_est": 14.2,
      "fall_detected": false, "immune_alert": false,
      "timestamp": 1720000000000
    },
    "sphereai": {
      "emotional_state": "calm", "stress_level": 0.21,
      "loneliness_index": 0.15, "cognitive_score": 0.94
    },
    "hollowrover": { "status": "idle", "battery": 87 },
    "hollowcore": { "ac_temp": 24, "fan": false },
    "entryguard": { "mode": "passive", "intruder_detected": false }
  }
}
```

2.5 Inter-Subsystem Communication Protocol

Subsystem communication follows a two-channel model. Cloud-mediated signals use Firebase Realtime Database listeners with target latency of 150ms. Hardware-to-hardware low-latency commands use direct MQTT over local Wi-Fi. HollowRover maintains a direct serial bridge to the ESP32 peripheral controllers at 115200 baud for motor and actuator commands requiring sub-10ms response times. GSM via SIM800L module serves as the fallback communication channel during Wi-Fi outages.

2.6 Multi-Timescale Inference Architecture

The AI inference pipeline operates across three simultaneous timescales:

- **Short-cycle (30s):** Runs on the normalized biometric feature vector for acute anomaly detection.
- **Medium-cycle (15m):** Updates emotional state and cortisol estimation models.
- **Long-cycle (24h):** Updates biological age clock, cognitive baseline deviation score, and personalized circadian rhythm model over a 30-day rolling window.

2.7 Fault Tolerance and Graceful Degradation

HollowRover carries a 5000mAh LiPo backup battery maintaining all critical operations during power outages and stores the last 24 hours of vital data locally on SD card. During Wi-Fi outages, HollowRover activates as a local Wi-Fi hotspot enabling Vital32 to continue transmitting. All five subsystems implement independent watchdog timers triggering reconnection attempts and guardian notifications upon detecting unresponsive nodes.

2.8 Security and Data Integrity

All biometric and personal data transmitted between subsystems and Firebase uses TLS encryption in transit. Face recognition profile data is stored locally on HollowRover and never transmitted to external servers. The intruder response protocol implements a mandatory human authorization step before any defensive action executes. Sensor readings are validated against physical plausibility bounds before entering the inference pipeline, filtering hardware faults from genuine biological anomalies.

3 Vital32: Complete Build Guide

Vital32 is the primary biosensing node of LifeSphere. It is built on the ESP32 WROOM-32 microcontroller and communicates with Firebase via Wi-Fi. This section provides complete wiring specifications, firmware architecture, and sensor calibration procedures sufficient to build and deploy a functional Vital32 unit from scratch.

3.1 Complete Component List

Component	Model / Part Number	Qty
Microcontroller	ESP32 WROOM-32 DevKit v1 (SPI/I2C/ADC/GPIO)	1
Heart Rate + SpO2	MAX30102 breakout board (I2C 0x57)	1
IMU (Fall + Tremor)	MPU6050 breakout board (I2C 0x68)	1
IR Temperature	MLX90614 breakout board (I2C 0x5A)	1
VOC + Humidity	BME688 breakout board (I2C 0x76)	1
UV Sensor	VEML6075 breakout board (I2C 0x10)	1
Grip Strength	FSR 402 force sensor + 10k resistor (ADC GPIO34)	1
Galvanic Skin Response	Grove GSR v1.0 or custom 2-electrode (ADC GPIO35)	1
Bioelectrical Impedance	AD5933 impedance converter (I2C 0x0D)	1
Near-IR Wound	850nm LED + BPW34 photodiode + opamp (ADC GPIO36)	1
OLED Display	SSD1306 0.96 inch 128x64 (I2C 0x3C)	1
LiPo Battery	3.7V 1000mAh LiPo + TP4056 charger	1
Charging Module	TP4056 with protection, USB-C input	1
3.3V Regulator	AMS1117-3.3 or ESP32 onboard VIN	1

3.2 Wiring and Pin Configuration

All I2C sensors share the SDA and SCL bus on GPIO21 and GPIO22 respectively. Each sensor has a unique I2C address allowing coexistence on one bus.

Listing 2: ESP32 Pin Assignments

```
// I2C Bus (shared):
GPIO21 (SDA) -> MAX30102, MPU6050, MLX90614, BME688, VEML6075, AD5933, SSD1306 SDA
GPIO22 (SCL) -> MAX30102, MPU6050, MLX90614, BME688, VEML6075, AD5933, SSD1306 SCL
3.3V -> VCC of all I2C modules | GND -> GND of all I2C modules

// Analog Inputs:
GPIO34 (ADC1_CH6) -> FSR 402: one leg to 3.3V, other to GPIO34 (10k pull-down to GND)
GPIO35 (ADC1_CH7) -> GSR sensor signal output
GPIO36 (ADC1_CH0) -> NIR photodiode amplifier output

// NIR Wound Healing Circuit:
GPIO32 (OUTPUT) -> 850nm LED anode via 100 ohm current-limiting resistor
BPW34 photodiode -> cathode to 3.3V, anode to inverting input of LM358
LM358 opamp -> configured as transimpedance amplifier, gain 100k
```

```
LM358 output -> GPIO36

// Power:
LiPo 3.7V -> TP4056 BAT+ BAT-
TP4056 OUT+ OUT- -> AMS1117-3.3 IN GND
AMS1117-3.3 OUT -> ESP32 3.3V pin
USB-C -> TP4056 IN+ IN- for charging
```

3.3 Firmware Architecture

The Vital32 firmware is written in Arduino C++ using the ESP32 Arduino Core. The main loop runs at 50Hz. Sensor reads are distributed across ticks using a scheduler to avoid I2C bus contention.

Listing 3: Vital32 Main Loop

```
void loop() {
  uint32_t t = millis();
  // Every tick (20ms):
  readMPU6050();          // accel + gyro at 50Hz for fall detection
  runFallDetector();
  // Every 1 second:
  if (t % 1000 == 0) {
    readMAX30102(); readMLX90614(); readGSR();
    readFSR(); calculateHRV(); runAnomalyScorer(); updateOLED();
  }
  // Every 5 seconds:
  if (t % 5000 == 0) {
    readBME688(); readVEML6075(); readNIR();
    readBIA(); runSeizureDetector(); // LSTM on rolling window
  }
  // Every 30 seconds:
  if (t % 30000 == 0) {
    runBioAgeModel(); writeToFirebase();
  }
}
```

3.4 Fall Detection Algorithm

Fall detection uses a two-stage algorithm on the MPU6050 accelerometer data. Stage 1 detects a freefall event by checking if the resultant acceleration magnitude drops below 0.4g. Stage 2 detects the impact by checking if the magnitude exceeds 2.5g within 500ms of the freefall. A zero-movement condition sustained for 3 seconds post-impact confirms the fall.

Listing 4: Fall Detection Implementation

```
bool detectFall(float ax, float ay, float az) {
  float magnitude = sqrt(ax*ax + ay*ay + az*az);
  if (magnitude < 0.4) {
    freefallTime = millis(); freefallDetected = true;
  }
  if (freefallDetected && magnitude > 2.5) {
    if (millis() - freefallTime < 500) {
      impactDetected = true; impactTime = millis();
    }
  }
  if (impactDetected && magnitude < 1.1) {
    if (millis() - impactTime > 3000) {
      freefallDetected = false; impactDetected = false;
      return true; // confirmed fall
    }
  }
  return false;
}
```

```
}
```

3.5 HRV Calculation

Heart Rate Variability is computed from the RR interval series extracted from the MAX30102 PPG signal. RMSSD is computed over a 2-minute rolling window of RR intervals. LF/HF ratio is computed via Welch PSD on the RR series.

Listing 5: RMSSD Calculation

```
float calculateRMSSD(float* rr_intervals, int n) {
    float sum_sq_diff = 0.0;
    for (int i = 0; i < n-1; i++) {
        float diff = rr_intervals[i+1] - rr_intervals[i];
        sum_sq_diff += diff * diff;
    }
    return sqrt(sum_sq_diff / (n - 1));
}
// LF band: 0.04-0.15 Hz | HF band: 0.15-0.40 Hz
// Computed using arduinoFFT library on 256-sample RR window
```

4 HollowRover: Complete Build Guide

HollowRover is the autonomous physical response unit of LifeSphere. It runs ROS2 Humble on a D-Robotics RDK X5 (4GB) and uses a dual-layer navigation architecture combining SLAM and UWB precision localization. The drivetrain employs a four-wheel rocker suspension with spring-loaded pivots and a rear differential bar for maintained ground contact over uneven surfaces. A 6-DOF robotic arm provides caregiving assistance and passive security barrier functionality.

4.1 Complete Component List

Component	Model / Part Number	Qty
Main Computer	D-Robotics RDK X5 4GB (8-core Cortex-A55, 10 TOPS NPU, Wi-Fi 6/BT 5.4)	1
Drivetrain MCU	ESP32 WROOM-32 DevKit v1 (UART to RDK X5, SPI, PWM)	1
Manipulator MCU	ESP32 WROOM-32 DevKit v1 (UART to RDK X5, I2C, PWM)	1
LiDAR	STL-19P TOF LiDAR (innomaker), 12m range, 5000Hz, 60000Lux anti-glare	1
IMU	MPU9250 9-axis (I2C 0x68 to Drivetrain ESP32)	1
Ultrasonic x4	HC-SR04 (GPIO on Drivetrain ESP32)	4
Thermal Camera	MLX90640 32x24 (I2C 0x33 to RDK X5)	1
RGB Camera	USB 1080p wide-angle (USB to RDK X5)	1
UWB Anchor x2	Decawave DW1000 module (SPI to Drivetrain ESP32)	2
Motor Driver	L298N dual H-bridge (GPIO PWM from Drivetrain ESP32)	1
DC Motors x4	12V 200RPM gear motor with encoder	4
Rocker Suspension	Spring-loaded pivot per rocker arm + rear differential bar	1 set
Robotic Arm Servos	MG996R metal gear (base, shoulder, elbow, wrist pitch, wrist roll)	5
Gripper Servo	MG90S (gripper)	1
Arm ToF Sensor	TF-Luna single-point ToF 0.2–8m (UART/I2C to Manipulator ESP32)	1
Main Battery	LiPo 11.1V 5000mAh (3S) with XT60 connector	1
Medication Dispenser	4x SG90 servo + pill compartments (PWM from Manipulator ESP32)	1
Water Spray	5V mini peristaltic pump + 100ml tank (GPIO relay from Manipulator ESP32)	1

4.2 ESP32 Peripheral Controller Wiring

The single ESP32 peripheral controller has been split into two independent boards to isolate timing-critical UWB ranging from servo PWM jitter contention.

Drivetrain Controller (ESP32 #1)

Listing 6: Drivetrain ESP32 Pin Assignments

```
// Motor Driver (L298N):
GPIO25 (PWM) -> L298N ENA | GPIO26 (PWM) -> L298N ENB
GPIO27 -> L298N IN1 | GPIO14 -> L298N IN2
GPIO12 -> L298N IN3 | GPIO13 -> L298N IN4

// Ultrasonic Sensors (HC-SR04):
GPIO5 TRIG1 / GPIO18 ECH01 -> Front
GPIO19 TRIG2 / GPIO21 ECH02 -> Left
GPIO22 TRIG3 / GPIO23 ECH03 -> Right
GPIO4 TRIG4 / GPIO15 ECH04 -> Rear

// UWB Modules (DW1000) x2:
GPIO32 (SPI SCK) -> DW1000 SCK | GPIO33 (MOSI) -> DW1000 MOSI
GPIO34 (MISO) -> DW1000 MISO | GPIO35 (CS1) -> DW1000 #1 CS
GPIO2 (CS2) -> DW1000 #2 CS

// UART to D-Robotics RDK X5:
GPIO1 (TX) -> RDK X5 UART RX | GPIO3 (RX) -> RDK X5 UART TX
Baud rate: 115200
```

Manipulator Controller (ESP32 #2)

Listing 7: Manipulator ESP32 Pin Assignments

```
// Robotic Arm Servos (MG996R x5 + MG90S x1):
GPIO13 -> Base rotation (MG996R)
GPIO14 -> Shoulder (MG996R)
GPIO27 -> Elbow (MG996R)
GPIO26 -> Wrist pitch (MG996R)
GPIO25 -> Wrist roll (MG996R)
GPIO33 -> Gripper (MG90S)

// Medication Dispenser Servos (SG90 x4):
GPIO16 -> Servo 1 (Slot A) | GPIO17 -> Servo 2 (Slot B)
GPIO18 -> Servo 3 (Slot C) | GPIO19 -> Servo 4 (Slot D)

// TF-Luna ToF Sensor (UART):
GPIO4 (TX) -> TF-Luna RX | GPIO5 (RX) -> TF-Luna TX

// Water Pump Relay:
GPIO12 -> Relay IN (peristaltic pump)

// UART to D-Robotics RDK X5 (second serial port):
GPIO9 (TX) -> RDK X5 UART1 RX | GPIO10 (RX) -> RDK X5 UART1 TX
Baud rate: 115200
```

4.3 ROS2 Package Structure

HollowRover runs ROS2 Humble Hawksbill on Ubuntu 22.04 Server (64-bit) on the D-Robotics RDK X5, which provides native ROS2 support. The dedicated 10 TOPS NPU/BPU allows SphereAI's AI inference (Whisper STT, DistilBERT sentiment analysis) to run in parallel with ROS2 Nav2/SLAM on separate silicon, eliminating CPU

contention that would occur on a general-purpose SoC. The complete ROS2 workspace is structured as follows:

Listing 8: ROS2 Workspace Structure

```
hollowrover_ws/
src/
  hollowrover_bringup/      <- Launch files for full system
  hollowrover_description/  <- URDF robot model
  hollowrover_navigation/   <- Nav2 configuration
  hollowrover_protocols/    <- Biomarker-triggered action nodes
  hollowrover_hardware/     <- ESP32 serial bridge nodes (x2)
  hollowrover_firebase/     <- Firebase state subscriber
  hollowrover_arm/          <- 6-DOF arm control node
```

4.4 SLAM Configuration

HollowRover uses SLAM Toolbox in lifelong mapping mode with the STL-19P TOF LiDAR providing scan data at 5000Hz ranging frequency with $\pm 10\text{mm}$ accuracy across a 12m radius. The TOF-based measurement principle ensures that accuracy does not degrade with distance, unlike triangulation-based LiDARs. The 60000Lux anti-glare rating prevents scan corruption in sunlit indoor environments near windows. The critical Nav2 parameters configure the controller frequency at 20.0Hz, max forward speed of 0.3m/s, and a safety inflation radius of 0.35m around obstacles.

4.5 UWB Localization

Two DW1000 UWB anchor modules are mounted at known fixed positions. The UWB tag on the Vital32 wearable broadcasts ranging packets. Trilateration is used when both anchors have valid range measurements.

Listing 9: UWB Trilateration Algorithm

```
def trilaterate(ax, ay, rA, bx, by, rB):
    d = sqrt((bx-ax)**2 + (by-ay)**2)
    a = (rA**2 - rB**2 + d**2) / (2 * d)
    h = sqrt(max(rA**2 - a**2, 0))
    x = ax + a*(bx-ax)/d
    y = ay + a*(by-ay)/d
    x1 = x + h*(by-ay)/d
    y1 = y - h*(bx-ax)/d
    return x1, y1 # take solution closer to last known position
```

4.6 Drivetrain: Rocker Suspension

HollowRover employs a four-wheel rocker suspension system replacing a flat rigid wheel mount. Each side features a rocker arm with a spring-loaded pivot, and a rear differential bar links the left and right rocker assemblies. The differential bar maintains chassis-level balance across uneven terrain, while the spring-loaded pivots maintain independent wheel-ground contact over bumps, doorway lips, and floor transitions. This passive suspension improves traction and stability compared to the original rigid-mount design without adding active control complexity.

4.7 HollowRover Robotic Arm

Mechanical Specification

HollowRover is equipped with a 6-DOF robotic arm mounted on the chassis:

- **Joint 1 — Base Rotation:** MG996R metal gear servo
- **Joint 2 — Shoulder:** MG996R metal gear servo
- **Joint 3 — Elbow:** MG996R metal gear servo
- **Joint 4 — Wrist Pitch:** MG996R metal gear servo
- **Joint 5 — Wrist Roll:** MG996R metal gear servo
- **Joint 6 — Gripper:** MG90S micro servo

Dual-Purpose Design

The arm serves two distinct functions:

(a) **Caregiving Assist:** The arm retrieves dropped medication or objects from the floor, and hands items (medication, water, personal items) to the user at an accessible height. This reduces the need for the elderly user to bend down or reach for objects.

(b) **Security Deterrent — Passive Barrier:** When EntryGuard’s face recognition pipeline (Section 5.2, DeepFace) flags an unregistered individual at a monitored chokepoint (e.g., a hallway leading to valuables or sensitive rooms), the arm swings to a blocking position across the pathway. The arm serves as a passive physical barrier that obstructs the intruder’s path.

Mechanical Constraint — LiDAR Occlusion

The arm rests in a folded/stowed position below the STL-19P LiDAR’s scan plane during normal navigation. It only rises to the blocking position when HollowRover is stationary in security-response mode. Navigation and defense states are mutually exclusive: the rover must be stationary before the arm deploys, so no LiDAR occlusion conflict occurs during active navigation.

Trigger Sensor and Detection Pipeline

A TF-Luna single-point ToF sensor (0.2–8m range, UART/I2C interface) is mounted at the designated chokepoint. When the ToF sensor detects a person’s presence, the RGB camera feed is routed into the existing DeepFace recognition pipeline. If the detected face is not matched against registered profiles, the security response is triggered.

Safety Constraint

The robotic arm is a passive physical barrier only. It makes no contact with any person, performs no grasping, restraining, or striking action against intruders, and initiates no autonomous defensive decision. The detection event triggers three simultaneous responses:

1. The arm moves to block the pathway (passive barrier only).
 2. An audible alarm is activated.
 3. An instant push notification with live camera feed is sent to the guardian’s phone.
- All escalation decisions beyond these three actions require explicit human authorization, consistent with the mandatory-human-authorization rule stated in Section 2.8.

5 SphereAI: Implementation Guide

SphereAI is the affective computing layer of LifeSphere. It implements a TriAgent architecture with three concurrent processing agents fused by a meta-classifier. The Personality Mirror feature has been removed from this version on ethical and privacy grounds; SphereAI learns preferences to improve interaction quality but does not build a deep personality model that could be replicated or shared.

5.1 Software Dependencies

Listing 10: SphereAI Python Dependencies

```
pip install openai-whisper      # Speech-to-text (Language Agent)
pip install librosa            # Audio feature extraction (Voice Agent)
pip install transformers torch # NLP sentiment + coherence
pip install scikit-learn       # Meta-classifier
pip install firebase-admin     # Firebase SDK
pip install numpy scipy        # Signal processing
pip install sounddevice        # Microphone capture
pip install pyttsx3            # Text-to-speech output
pip install luma.oled          # SSD1306 OLED display
```

5.2 TriAgent Fusion Logic

The MetaClassifier applies dynamic weights based on the prior emotional state to fuse outputs from the Language, Voice, and Biology agents.

Listing 11: SphereAI Meta-Classifier

```
class MetaClassifier:
    WEIGHTS = {
        'calm':      {'language': 0.4, 'voice': 0.3, 'biology': 0.3},
        'stressed':  {'language': 0.2, 'voice': 0.3, 'biology': 0.5},
        'grief':     {'language': 0.3, 'voice': 0.4, 'biology': 0.3},
        'isolated':  {'language': 0.2, 'voice': 0.2, 'biology': 0.6},
        'cognitive': {'language': 0.5, 'voice': 0.3, 'biology': 0.2},
    }

    def fuse(self, lang_out, voice_out, bio_out, prior_state):
        w = self.WEIGHTS.get(prior_state, self.WEIGHTS['calm'])
        scores = {}
        for state in EMOTIONAL_STATES:
            scores[state] = (
                w['language'] * lang_out.get(state, 0) +
                w['voice'] * voice_out.get(state, 0) +
                w['biology'] * bio_out.get(state, 0)
            )
        return max(scores, key=scores.get), scores
```

6 EntryGuard and HollowCore: Implementation Guide

6.1 EntryGuard Hardware per Node

Each EntryGuard node runs an ESP32 WROOM-32 equipped with a PIR sensor (HC-SR501), ultrasonic sensor (HC-SR04), fire sensor (KY-026), and gas sensor (MQ-2). All sensor events are published to Firebase.

Listing 12: EntryGuard Node Firmware

```
void loop() {
  bool motion = digitalRead(PIR_PIN);
  int fire = analogRead(FIRE_PIN);
  int gas = analogRead(GAS_PIN);
  long dist = measureUltrasonic();
  if (motion && systemMode == ACTIVE)
    publishAlert("motion", dist, nodeID);
  if (fire > FIRE_THRESHOLD) {
    publishAlert("fire", fire, nodeID); triggerBuzzer();
  }
  if (gas > GAS_THRESHOLD) {
    publishAlert("gas", gas, nodeID); triggerBuzzer();
  }
  delay(100);
}
```

6.2 HollowCore Wiring

HollowCore uses an ESP32 to control relay modules for appliance operation. It listens to Mosquitto MQTT broker running on the D-Robotics RDK X5 and maps Firebase state variables to relay triggers.

Listing 13: HollowCore Pin Assignments

```
GPI026 -> Relay 1 IN (Living room light)
GPI027 -> Relay 2 IN (Bedroom light)
GPI014 -> Relay 3 IN (AC / fan)
GPI012 -> Relay 4 IN (TV power)
GPI013 -> Relay 5 IN (Main door lock solenoid)
// MQTT Topic: lifesphere/hollowcore/command
// Payload: {"device": "light_1", "state": true}
```

6.3 HollowCore Financial Anomaly Detection & Biometric Security

HollowCore extends its monitoring beyond the physical environment to the user's financial and digital security. By integrating with the user's banking and subscription services via secure APIs, HollowCore scans incoming transactions and recurring subscriptions in real-time.

Transaction Security Features

- **Anomaly Detection:** Utilizes an Isolation Forest model trained on the user's historical spending habits to flag unrecognized, out-of-pattern, or suspicious transactions.
- **Smart Notification:** Instantly alerts the user via the LifeSphere app and HollowRover OLED, providing transaction details and a one-tap option to freeze the card or dispute the charge.

- **Biometric Transaction Authorization:** For the user to authorize any new high-risk transaction or subscription manually, HollowCore mandates multi-factor biometric verification. This requires facial recognition (via EntryGuard/HollowRover camera), fingerprint scan, and proximity confirmation from the Vital32 wearable, ensuring maximum financial security.

7 Firebase Backend Architecture

The Firebase Realtime Database serves as the central nervous system of LifeSphere. All five subsystems read from and write to a single JSON tree.

7.1 Firebase Project Setup

1. Create a new project named LifeSphere at console.firebase.google.com.
2. Enable Realtime Database. Choose region asia-southeast1 (Singapore).
3. Set database rules to authenticated-only access.
4. Download the service account JSON for Python SDK access.
5. Get the Web API key for ESP32 REST API access.

7.2 ESP32 Firebase Write (Vital32/EntryGuard)

Listing 14: ESP32 Firebase Write

```
#include <FirebaseESP32.h>
#include <WiFi.h>

#define FIREBASE_HOST "your-project.firebaseio.com"
#define FIREBASE_AUTH "your-database-secret"

FirebaseData fbdo;

void writeToFirebase(SensorData data) {
    Firebase.setFloat(fbdo, "/lifesphere/vital32/heart_rate", data.hr);
    Firebase.setFloat(fbdo, "/lifesphere/vital32/spo2", data.spo2);
    Firebase.setBool (fbdo, "/lifesphere/vital32/fall_detected", data.fall);
}
```

7.3 Python Firebase Read (SphereAI/HollowRover)

Listing 15: Python Firebase Read

```
import firebase_admin
from firebase_admin import credentials, db

cred = credentials.Certificate('serviceAccountKey.json')
firebase_admin.initialize_app(cred, {
    'databaseURL': 'https://your-project.firebaseio.com'
})

ref = db.reference('/lifesphere/vital32')

def on_vital32_change(event):
    data = event.data
    if data.get('fall_detected'):
        dispatch_fall_protocol()
    if data.get('anomaly_score', 0) > 0.9:
        dispatch_emergency_protocol()
    if data.get('seizure_prob', 0) > 0.85:
        dispatch_seizure_protocol()

ref.listen(on_vital32_change)
```

8 AI Model Pipeline: Training and Deployment

All AI models in LifeSphere are trained offline and deployed as TensorFlow Lite models on the respective hardware. The AI pipeline is the brain of the ecosystem, transforming raw sensor data into actionable medical and emotional insights.

Impact on Humanity

The AI Pipeline represents a paradigm shift in personal healthcare. By utilizing multi-timescale inference (short, medium, and long-cycle), the pipeline learns the user's unique biological baseline. This means LifeSphere doesn't just compare the user to a generic population—it compares the user to their own healthy state, detecting subtle deviations that precede critical events, thus democratizing predictive healthcare.

8.1 Isolation Forest Anomaly Detector

The Isolation Forest model runs on Vital32 (via TFLite) and serves as the primary real-time anomaly detector. It is trained on the user's own biometric data collected during the first 30 days of normal use.

Listing 16: Isolation Forest Training

```
from sklearn.ensemble import IsolationForest
import numpy as np

def train_isolation_forest(baseline_data):
    model = IsolationForest(
        n_estimators=100, max_samples='auto',
        contamination=0.05, random_state=42
    )
    model.fit(baseline_data)
    return model

def get_anomaly_score(model, sample):
    raw = model.score_samples([sample])[0]
    return 1 - (raw + 0.5) # normalize to 0-1
```

8.2 Pre-Seizure LSTM

The pre-seizure detection LSTM is trained on the CHB-MIT Scalp EEG Dataset and the Epilepsy iEEG Prediction dataset. The model takes 5-minute rolling windows of HRV, accelerometer, and skin temperature features as input.

Listing 17: Pre-Seizure LSTM Model

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

def build_seizure_model():
    model = Sequential([
        LSTM(64, input_shape=(300, 5), return_sequences=True),
        Dropout(0.3),
        LSTM(32, return_sequences=False),
        Dropout(0.3),
        Dense(16, activation='relu'),
        Dense(1, activation='sigmoid') # seizure probability
    ])
    model.compile(optimizer='adam',
```

```

        loss='binary_crossentropy',
        metrics=['accuracy', 'AUC'])
    return model

```

8.3 SphereAI Emotional State Classifier

The Voice Agent utilizes Librosa to extract MFCC, pitch, energy, tempo, and zero-crossing rate. The Language Agent uses a HuggingFace DistilBERT model for sentiment analysis. Cognitive assessment computes unique word ratio, topic repetition, and semantic coherence.

8.4 Model Performance Summary

Model	Architecture	Output	Threshold
Anomaly Detector	Isolation Forest (100 trees)	Score 0–1	0.7 / 0.9
Pre-Seizure LSTM	2×LSTM (64,32) + Dense	Prob 0–1	0.85
Emotional Classifier	Weighted 3-stream fusion	State category	Highest
Cortisol Estimator	Gradient Boost (200 trees)	ng/dL est.	15m update
Biological Age Clock	Ensemble (RF + XGBoost)	Age in years	Daily update
Cognitive Tracker	Statistical drift (CUSUM)	Deviation score	2-sigma

9 Mathematical Foundations

9.1 Biometric Signal Processing

Heart Rate Variability is computed from the RR interval time series extracted from the MAX30102 PPG waveform. The primary HRV metrics are:

$$\text{RMSSD} = \sqrt{\frac{1}{N-1} \sum_{i=1}^{N-1} (RR_{i+1} - RR_i)^2} \quad (1)$$

$$\text{LF/HF} = \frac{P_{LF} [0.04-0.15 \text{ Hz}]}{P_{HF} [0.15-0.40 \text{ Hz}]} \quad (2)$$

$$C_{\text{est}} = f(\text{RMSSD}, \text{GSR}_{\text{baseline}}, T_{\text{skin}}, \text{sleep_efficiency}) \quad (3)$$

9.2 Isolation Forest Anomaly Score

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}} \quad (4)$$

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \quad \text{where} \quad H(n) = \ln(n) + 0.5772 \quad (5)$$

Scores $s(x, n) > 0.7$ trigger a soft alert. Scores > 0.9 trigger an emergency protocol dispatch.

9.3 Medication Adherence Entropy

$$H(X) = - \sum_i p(x_i) \log_2(p(x_i)) \quad (6)$$

High entropy indicates irregular adherence. Low entropy indicates consistent adherence.

9.4 UWB Trilateration

$$(x - a_x)^2 + (y - a_y)^2 = r_A^2 \quad (7)$$

$$(x - b_x)^2 + (y - b_y)^2 = r_B^2 \quad (8)$$

$$a = \frac{r_A^2 - r_B^2 + d^2}{2d} \quad \text{where} \quad d = \text{dist}(\text{Anchor}_A, \text{Anchor}_B) \quad (9)$$

$$x = a_x + \frac{a(b_x - a_x)}{d}, \quad y = a_y + \frac{a(b_y - a_y)}{d} \pm h \quad (10)$$

10 Prescription OCR and Automated Medication Reminders

A critical challenge in elderly medication management is the initial setup burden: manually entering every medicine name, dosage, and timing is error-prone and discourages adoption. LifeSphere addresses this through a one-time prescription photograph workflow. The user photographs their prescription once, and the system automatically extracts and creates a complete medication schedule. Manual entry remains available as an alternative, and all extracted data is fully editable to account for handwriting recognition errors.

Feature Overview

- User photographs prescription once using HollowRover camera or mobile app.
- Vision AI model extracts: medicine name, dosage, frequency, duration, instructions.
- Medication schedule is automatically created in Firebase and synced to HollowRover.
- User reviews and edits extracted data via HollowCore app before confirming.
- HollowRover begins autonomous medication delivery on the confirmed schedule.

10.1 System Overview

The prescription OCR pipeline runs on the D-Robotics RDK X5 and uses a combination of image preprocessing, an OCR engine, and a large language model for structured data extraction. The pipeline is designed to handle the notorious variability of doctor handwriting by using an LLM to interpret context rather than relying on raw OCR output alone.

10.2 OCR Pipeline and Data Extraction

Listing 18: Prescription OCR Pipeline

```
import pytesseract, cv2, base64, requests
from PIL import Image
import firebase_admin
from firebase_admin import db

def preprocess_image(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    denoised = cv2.fastNlMeansDenoising(gray, h=10)
    _, thresh = cv2.threshold(denoised, 0, 255,
                              cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    return thresh

def extract_text_ocr(image_path):
    processed = preprocess_image(image_path)
    raw_text = pytesseract.image_to_string(processed,
                                          config='--psm 6 --oem 3')

    return raw_text

def parse_prescription_with_llm(raw_text):
    prompt = f"""Extract medication details from this prescription text.
    Return ONLY valid JSON in this exact format:
    {{
      "medications": [{
        "name": "medicine name", "dosage": "500mg",
```

```

        "frequency": "twice daily", "times": ["08:00", "20:00"],
        "duration_days": 7, "instructions": "take with food",
        "confidence": 0.92
    }]}
    }
    Prescription text: {raw_text}"""
    response = call_llm_api(prompt) # Gemini Flash or GPT-4o mini
    return parse_json_response(response)

def create_medication_schedule(medications, user_id):
    ref = db.reference(f'/lifesphere/medication_schedule/{user_id}')
    for med in medications:
        ref.push({
            'name': med['name'], 'dosage': med['dosage'],
            'times': med['times'],
            'duration_days': med['duration_days'],
            'instructions': med['instructions'],
            'confirmed': False, 'active': False,
        })

```

10.3 Reminder Schedule Integration

Once the user confirms the extracted schedule via the HollowCore app, the `medication_delivery.py` ROS2 node subscribes to the confirmed schedule and begins autonomous delivery. Each medication slot in the HollowRover dispenser is mapped to a specific schedule entry. The pressure sensor beneath each slot records when medication is removed and updates the adherence log in Firebase.

10.4 Firebase Medication Schedule Schema

Listing 19: Firebase Medication Schedule Schema

```

/lifesphere/medication_schedule/{user_id}/{med_id}/
name: "Metformin"
dosage: "500mg"
times: ["08:00", "13:00", "20:00"]
duration_days: 30
instructions: "take with meal"
slot: 1 // HollowRover dispenser slot number
confirmed: true // user confirmed OCR extraction
active: true // currently in delivery rotation
adherence_log: { // updated by pressure sensor
  "2024-07-14": {"08:00": true, "13:00": true, "20:00": false}
}

```

11 Absence Mode and Autonomous Household Management

When the primary occupant is away from home for an extended period, LifeSphere transitions into Absence Mode. In this mode, the system continues to maintain the household autonomously by learning from the occupant's historical activity patterns, receiving explicit task instructions from the occupant or designated guardians, and executing household maintenance routines using HollowRover. This transforms LifeSphere from a reactive caregiving system into a proactive household management system during occupant absence.

Absence Mode Capabilities

- Learns daily household routines from 30 days of historical activity data.
- Receives task instructions from occupant via HollowCore app or voice message.
- HollowRover executes physical tasks: plant watering, checking appliances, patrols.
- EntryGuard enters full active security mode automatically on absence detection.
- Daily status report sent to occupant: photos, task completion, security events.
- Returns to normal caregiving mode automatically when occupant returns.

11.1 Absence Detection

Absence is detected through a combination of signals: the occupant explicitly activating AbsenceMode via the app, or automatic detection when Vital32 wearable shows no activity for 4+ hours while EntryGuard records a departure event at the front door. The system confirms absence before activating to avoid false triggers from deep sleep or illness.

12 Remote Guardian and Digital Safety Integration

LifeSphere extends its safety umbrella beyond the primary elderly user to protect children and other vulnerable family members. Through the LifeSphere mobile app, parents and guardians can remotely configure the ecosystem, add dependent users, and instruct the AI on specific safety parameters to monitor.

12.1 Remote User Configuration

A guardian (e.g., a father working abroad) can add a child as a dependent user within the app. The guardian defines specific instructions for SphereAI and HollowCore, such as monitoring for signs of distress, ensuring the child takes medication, or maintaining a secure perimeter via EntryGuard. These instructions are parsed by SphereAI's language agent and converted into active monitoring protocols.

12.2 AI-Powered Content Safety

A core component of the Remote Guardian feature is digital safety. Guardians can instruct the AI to restrict and monitor the child's digital devices connected to the HollowCore Wi-Fi network. HollowCore employs network-level filtering and SphereAI analyzes activity patterns to detect and block inappropriate or harmful content in real-time.

Digital Safety Features

- **Network-Level Filtering:** HollowCore intercepts DNS requests on the home network, blocking access to adult, violent, or malicious websites based on a dynamically updated categorization database.
- **Smart Screen Time:** Guardians can set adaptive screen time limits. HollowCore can gradually throttle bandwidth or disable internet access to specific devices when limits are exceeded.
- **Inappropriate Content Alerts:** If SphereAI detects patterns suggesting exposure to harmful content (e.g., sudden emotional distress correlated with device usage), it alerts the guardian with context and a recommendation to review activity.

13 Full Bill of Materials

13.1 Vital32

Component	Model / Spec	Cost (USD)
Microcontroller	ESP32 WROOM-32 DevKit v1	\$7.50
Heart Rate + SpO2	MAX30102 breakout	\$5.50
IMU	MPU6050 breakout	\$2.80
IR Temperature	MLX90614 breakout	\$4.50
VOC + Humidity	BME688 breakout	\$11.00
UV Sensor	VEML6075 breakout	\$3.50
Grip Strength	FSR 402 + 10k Ω resistor	\$2.50
Galvanic Skin Response	Grove GSR v1.0	\$4.50
Bioelectrical Impedance	AD5933 impedance converter	\$7.50
Near-IR Wound	850nm LED + BPW34 + LM358	\$2.80
OLED Display	SSD1306 0.96" 128x64	\$3.50
LiPo Battery	3.7V 1000mAh	\$4.50
Charging Module	TP4056 with protection	\$1.80
3.3V Regulator	AMS1117-3.3	\$0.80
Miscellaneous	Wires, PCB, connectors	\$5.00
Vital32 Subtotal		\$67.70

13.2 HollowRover

Component	Model / Spec	Cost (USD)
Main Computer	D-Robotics RDK X5 (4GB), 10 TOPS NPU	\$99.00
Drivetrain MCU	ESP32 WROOM-32 #1	\$7.50
Manipulator MCU	ESP32 WROOM-32 #2	\$7.50
LiDAR	STL-19P TOF (innomaker), 12m, 5000Hz, 60kLux	\$65.00
IMU	MPU9250 9-axis	\$4.50
Ultrasonic Sensors	HC-SR04 ×4	\$8.00
Thermal Camera	MLX90640 32×24	\$19.00
RGB Camera	USB 1080p wide-angle	\$12.00
UWB Anchors	DW1000 ×2	\$20.00
Motor Driver	L298N dual H-bridge	\$4.50
DC Motors	12V 200RPM gear motor ×4	\$16.00
Rocker Suspension	Spring-loaded pivots + differential bar set	\$12.00
Chassis Frame	Aluminum frame, custom	\$15.00
Robotic Arm Servos	MG996R metal gear ×5	\$25.00
Gripper Servo	MG90S ×1	\$2.80
Arm ToF Sensor	TF-Luna single-point ToF	\$14.50
Medication Dispenser	SG90 servo ×4 + compartments	\$8.00
Water Spray	5V peristaltic pump + 100ml tank	\$5.00
Main Battery	LiPo 11.1V 5000mAh (3S) + XT60	\$24.00
Connectors & Wiring	XT60, JST, Dupont wires	\$4.00
HollowRover Subtotal		\$373.30

13.3 SphereAI

Component	Model / Spec	Cost (USD)
USB Microphone	USB condenser microphone	\$7.50
Mini Speaker	3W 4Ω mini speaker	\$4.50
SphereAI Subtotal		\$12.00

13.4 EntryGuard (×2 Nodes)

Component	Model / Spec	Cost (USD)
Microcontroller	ESP32 WROOM-32 ×2	\$15.00
PIR Sensor	HC-SR501 ×2	\$4.00
Ultrasonic Sensor	HC-SR04 ×2	\$4.00
Fire Sensor	KY-026 ×2	\$3.00
Gas Sensor	MQ-2 ×2	\$5.00
Buzzer	Active buzzer ×2	\$1.50
Enclosure & Wiring	3D-printed enclosure, wires ×2	\$4.00
EntryGuard Subtotal		\$36.50

13.5 HollowCore

Component	Model / Spec	Cost (USD)
Microcontroller	ESP32 WROOM-32	\$7.50
Relay Module	5-channel relay module	\$4.50
Enclosure & Wiring	3D-printed enclosure, wires	\$3.00
HollowCore Subtotal		\$15.00

13.6 System Total

Subsystem	Cost (USD)
Vital32	\$67.70
HollowRover	\$373.30
SphereAI	\$12.00
EntryGuard (×2)	\$36.50
HollowCore	\$15.00
LifeSphere Total	\$504.50

14 Conclusion

LifeSphere demonstrates that a fully integrated, AI-powered autonomous caregiving ecosystem is achievable at an accessible cost point using commercially available hardware and open-source software frameworks. By unifying health monitoring (Vital32), physical response (HollowRover), emotional intelligence (SphereAI), perimeter security (EntryGuard), and biology-responsive automation (HollowCore) around a single shared biological state backbone, the system achieves a level of proactive, multi-dimensional care that no combination of standalone devices can replicate.

The key technical contributions of this work include: (1) a multi-timescale AI inference pipeline that learns individual biological baselines rather than relying on population-level thresholds; (2) a ROS2-based autonomous mobile platform with dedicated NPU compute enabling parallel SLAM navigation and AI inference without CPU contention; (3) a trimodal emotional state classifier fusing language, voice, and biomarker signals with context-adaptive weighting; (4) a passive physical barrier system via a 6-DOF robotic arm that enforces security boundaries without making human contact; and (5) a prescription OCR pipeline that eliminates the medication setup barrier for elderly users.

LifeSphere does not require the user to interact with technology. It requires technology to respond to the user's biology. This principle—that the most vulnerable members of society should not need to learn complex interfaces to receive care—drives every architectural decision in this system. From the zero-interaction Vital32 wearable to the autonomous HollowRover dispatch protocols, the system is designed to disappear into the background of daily life while continuously protecting the user across medical, physical, emotional, and environmental dimensions.

Future work includes extending the biological state model with additional sensor modalities, integrating legged locomotion for multi-floor navigation, and conducting longitudinal user studies to validate predictive health model accuracy in real-world deployment conditions.

References

- [1] Liu, F.T., Ting, K.M., & Zhou, Z.H. (2008). Isolation Forest. *IEEE International Conference on Data Mining*, 413–422.
- [2] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [3] Devlin, J., Chang, M.W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT*, 4171–4186.
- [4] Radford, A., Kim, J.W., Xu, T., et al. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. *ICML*.
- [5] Bosse, T., Kester, L., & de Man, J. (2022). ROS2 Navigation: From SLAM to Nav2. *IEEE Robotics & Automation Magazine*, 29(3), 56–67.
- [6] Goldberger, A.L., Amaral, L.A.N., Glass, L., et al. (2000). PhysioBank, PhysioToolkit, and PhysioNet. *Circulation*, 101(23), e215–e220.
- [7] Shoeb, A.H. (2009). Application of Machine Learning to Epileptic Seizure Detection. *M.S. Thesis, MIT*.
- [8] Smith, S.W. (1997). *The Scientist and Engineer’s Guide to Digital Signal Processing*. California Technical Publishing.
- [9] Innomaker. (2024). STL-19P TOF LiDAR Datasheet. <https://www.innomaker.com>
- [10] D-Robotics. (2024). RDK X5 Developer Kit Technical Reference Manual. <https://www.d-robotics.cc>
- [11] Decawave. (2016). DW1000 User Guide. *Decawave Ltd*.
- [12] Google. (2024). Firebase Realtime Database Documentation. <https://firebase.google.com/docs/database>
- [13] Liu, W., Wen, Y., Yu, Z., & Yang, M. (2017). Large-Margin Softmax Loss for Convolutional Neural Networks. *ICML*, 507–516.
- [14] McFee, B., Raffel, C., Liang, D., et al. (2015). librosa: Audio and Music Signal Analysis in Python. *Proc. SciPy*, 18–24.
- [15] Tesseract OCR. (2024). Tesseract Open Source OCR Engine. <https://github.com/tesseract-ocr/tesseract>